

NLP Assignment 2

Word Embeddings & Character-Level Name Generation

Student : Jateen

Roll No. : B22CS026

Course : Natural Language Processing

Institute : IIT Jodhpur

Problem 1: IIT Jodhpur Word Embeddings (Word2Vec from scratch + Gensim)

Problem 2: Character-Level Indian Name Generation (Vanilla RNN | Bi-LSTM | Attention+RNN)

1.1 Dataset Preparation

► Sources Collected

- IIT Jodhpur official website: departments, academics, research, announcements, faculty profiles, campus life.
- Annual Reports (PDF): AR 2008–09 through AR 2012–13 — rich institutional and programmatic text.
- Institute Brochure (PDF, Mar 2025): comprehensive 36 KB of academic programme descriptions.
- Newsletter (PDF, May 2025): 63 KB of recent research highlights, faculty achievements, and institutional news.
- Senate Constitution (PDF, Jan 2026): formal governance document adding specialised academic vocabulary.
- Vision & Mission and History pages: foundational language of the institute.

► Preprocessing Pipeline

- Step 1 Data Collection — scraped 50 unique sources using Requests and BeautifulSoup4; PDFs downloaded and parsed with PyPDF2.
- Step 2 Boilerplate Removal — removed navigation menus, footers, headers using tag-level HTML decomposition.
- Step 3 Non-English Filtering — lines with less than 60% ASCII characters are discarded; removes Hindi/Sanskrit script.
- Step 4 Text Cleaning — stripped URLs, emails, citation numbers, LaTeX fragments, and non-informative symbols via regex.
- Step 5 Lowercasing — entire corpus lowercased for vocabulary consistency.
- Step 6 Tokenisation — NLTK sent_tokenize splits into sentences; word_tokenize produces word-level tokens.
- Step 7 Frequency Filtering — words with count < 2 are excluded from Word2Vec training as min_count=2.

1.2 Corpus Statistics

Metric	Value
Total documents (pages + PDFs)	50
Total sentences	5,228
Total tokens	153,804
Vocabulary size (unique words)	9,970
Corpus file size	0.937 MB (982,830 bytes)
Average sentence length	29.4 tokens

► Top-10 Words by Frequency

Rank	Word	Freq	Rank	Word	Freq
1	the	6,154	6	for	1,805
2	of	4,758	7	a	1,677
3	and	4,423	8	ug	1,014
4	in	2,181	9	iit	951
5	to	2,174	10	students	951

Problem 1: Word2Vec — Theory & Hyperparameter Experiments

1.3 Word2Vec Theory

► Continuous Bag-of-Words (CBOW)

CBOW predicts a target word from its surrounding context. Given context words $\{w_{t-k}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+k}\}$, the model averages their embeddings and passes them through a softmax output layer to predict w_t . The objective is to maximise: $J = \log P(w_t | \text{context})$. CBOW is faster to train and works better for frequent words, but it smooths over individual context words.

► Skip-Gram

Skip-gram inverts the CBOW objective: given a target word w_t , it predicts each surrounding context word. The objective is: $J = \sum \log P(w_{t+j} | w_t)$ for j in $[-k, k] \setminus \{0\}$. Skip-gram is slower to train but captures rare word relationships better since each (target, context) pair is a separate prediction. With negative sampling, training approximates the full softmax by optimising a binary classifier on k negative samples drawn from a noise distribution $P(w) \propto f(w)^{3/4}$.

► Why Negative Sampling?

Full softmax over a 9,970-word vocabulary is expensive for every training step. Noise-Contrastive Estimation / Negative Sampling (NEG) replaces it with a simpler binary classification: distinguish the true context word from k randomly-sampled noise words. This reduces per-step cost from $O(V)$ to $O(k)$, making training on large corpora tractable.

1.4 Hyperparameter Experiments (36 Total)

A grid search was performed over both architectures varying embedding dimension (50, 100, 200), context window size (3, 5, 7), and negative samples (5, 10). All models trained for 10 epochs with `min_count=2`.

Models evaluated by average cosine similarity of top-5 neighbours for a fixed set of query words.

Model	Embed Dim	Window	Neg Samples	Avg Similarity
CBOW (best)	200	5	5	0.8550
CBOW	100	5	5	0.8513
CBOW	50	7	5	0.8482
Skip-gram(best)	50	3	5	0.7347
Skip-gram	50	7	10	0.7400
Skip-gram	200	3	5	0.6627

Best CBOW: `dim=200, window=5, neg=5` (`avg_sim=0.8550`). Best Skip-gram: `dim=50, window=3, neg=5` (`avg_sim=0.7347`). CBOW consistently outscored Skip-gram on the average-similarity metric because the metric rewards tight clusters; Skip-gram's strength is relational/analogy accuracy, not neighbourhood compactness.

1.5 Semantic Analysis Results

► Nearest Neighbours (Skip-gram, dim=200)

research → activities, proposal, interdisciplinary, development, areas
engineering → metallurgical, bioengineering, materials, computer, bioscience
student → candidate, assistantship, advise, until, visiting
examination → comprehensive, viva-voce, make-up, examiners, examinations
phd → mtech, 22.6, m.tech./m.tech.-ph.d., dual, governing

► Analogies Resolved

- ✓ UG : B.Tech :: PG : M.Tech (model correctly identifies postgraduate degree)
- ✓ semester : examination :: thesis : defense (viva-voce / defense as the terminal assessment)
- ✓ student : learning :: faculty : teaching

1.6 From-Scratch Word2Vec — PyTorch Implementation

A complete Word2Vec implementation was built without any high-level NLP libraries, using only PyTorch. The implementation includes:

- Vocabulary class — builds word-to-index mapping, computes unigram frequency distribution for negative sampling: $P(w) \propto f(w)^{(3/4)}$.
- CBOWDataset / SkipGramDataset — PyTorch Dataset classes that create (context, target, negatives) training tuples.
- CBOWModel / SkipGramModel — nn.Module with two embedding tables (input and output). Forward pass computes negative-sampling binary cross-entropy loss directly.
- Training loop — Adam optimiser, 50 epochs, gradient clipping. CBOW: 14,564 pairs. Skip-gram: 212,518 pairs (more pairs as each target-context pair is separate).

Scratch vs. Gensim Comparison (Nearest Neighbours for 'engineering')

Scratch CBOW computer (0.938), metallurgical (0.920), electrical (0.913), materials (0.902)
Gensim CBOW science (0.967), computer (0.958), metallurgical (0.949), electrical (0.947)
Scratch SG metallurgical (0.607), bridge (0.593), bioengineering (0.570)
Gensim SG metallurgical (0.960), bioengineering (0.959), materials (0.954)

Observation: Scratch CBOW matches Gensim's top choices almost identically despite lower absolute similarities, validating the implementation. Gensim benefits from optimised C code with subsampling and hierarchical softmax enhancements. For the analogy thesis : examination :: thesis : ?, scratch CBOW returns 'submit, defend, synopsis' — arguably more meaningful answers than Gensim's 'src, reported, dugc'.

1.7 300-Dimensional Embedding Vector for 'student'

Trained using Word2Vec Skip-gram (vector_size=300, window=5, negative=5, min_count=2, epochs=20, seed=42) on the full 0.937 MB IITJ corpus:

student —

-0.4817,	0.3603,	0.0493,	0.4250,	-0.3770,	-0.0673,	0.0594,	0.3550,	0.4441,	0.0108,	0.4734,	0.2026,
-0.0270,	0.5809,	-0.2244,	-0.5872,	0.0789,	-0.1344,	-0.1896,	-0.5505,	0.3547,	0.5694,	-0.2025,	
0.3512,	0.2020,	-0.0790,	-0.4439,	-0.6036,	0.4092,	0.0152,	-0.8031,	-0.1061,	0.2068,	-0.0710,	
0.1819,	-0.1978,	0.1547,	0.2107,	-0.0920,	-0.2565,	0.3691,	-0.9547,	-0.2578,	-0.0465,	0.6128,	
-0.1441,	0.2610,	-0.0176,	-0.5609,	0.1823,	-0.1211,	-0.2968,	-0.2824,	-0.4080,	0.1438,	-0.1003,	
0.1738,	-0.1842,	0.6324,	0.2406,	0.1792,	0.3348,	0.2112,	-0.3231,	-0.0153,	0.3718,	-0.3385,	0.1989,
0.0410,	1.1182,	0.2469,	0.2355,	-0.7243,	-0.1846,	-0.7783,	-0.2690,	-0.2737,	0.1185,	-0.1831,	
0.0040,	-0.3267,	0.3263,	0.1461,	-0.3609,	-0.2161,	-0.4403,	0.5465,	-0.3295,	-0.0014,	-0.1480,	
-0.3047,	-0.0644,	-0.4163,	-0.2091,	-0.4770,	-0.1277,	-0.3629,	0.3530,	0.2578,	0.4835,	0.2564,	
0.3275,	0.7638,	-0.5899,	-0.1397,	-0.1678,	0.3567,	-0.2041,	-0.4595,	-0.1959,	-0.0501,	0.2504,	
-0.0106,	-0.1677,	-0.4346,	0.0239,	0.1260,	0.1062,	-0.7511,	0.0171,	0.2365,	0.0898,	-0.6174,	
-0.2059,	-0.2231,	0.0275,	-0.5134,	0.5352,	0.0127,	0.0232,	-0.1253,	-0.1755,	0.2097,	-0.4404,	
-0.4178,	-0.0805,	-0.3315,	-0.0120,	-0.2659,	-0.5955,	0.0000,	0.1635,	-0.2164,	-0.0874,	-0.1708,	
-0.0451,	0.7730,	-0.0028,	0.0360,	-0.3441,	0.0187,	-0.3389,	0.2444,	0.3140,	-0.2221,	-0.4685,	
-0.4509,	0.3447,	0.2380,	-0.0195,	-0.1323,	-0.0985,	-0.0388,	0.0161,	-0.5808,	0.0477,	0.2680,	
-0.2245,	0.5469,	0.2117,	-0.5573,	0.4815,	0.3396,	-0.0683,	0.0381,	-0.5191,	0.0503,	-0.1327,	
-0.0704,	0.1641,	-0.1939,	-0.2861,	-0.6366,	0.1738,	-0.6682,	-0.1642,	-0.0663,	-0.2858,	-0.1228,	
-0.1536,	-0.0461,	-0.6755,	-0.1673,	-0.1956,	-0.3070,	-0.3313,	-0.1935,	0.1770,	0.0497,	-0.4135,	
0.5211,	-0.6660,	-0.3345,	-0.0580,	-0.5002,	-0.3094,	-0.1044,	-0.2882,	0.0873,	0.3455,	-0.2908,	
0.1523,	0.1844,	0.3205,	0.2177,	0.3667,	-0.1245,	-0.2883,	0.5046,	0.6984,	-0.4036,	0.1387,	0.5568,
0.0524,	0.2946,	0.1458,	0.5299,	-0.0657,	-0.7453,	0.0522,	0.3259,	0.0074,	0.2894,	0.0353,	-0.5590,
0.0846,	0.2497,	-0.2185,	0.2897,	-0.1043,	-0.5683,	0.3164,	-0.0470,	0.0958,	-0.2009,	-0.5172,	
-0.0821,	-0.0654,	0.6564,	-0.8608,	0.1632,	0.1346,	-0.0572,	0.0744,	-0.2215,	-0.0736,	0.1789,	
-0.0172,	0.0034,	0.3277,	0.4270,	-0.2283,	-0.4266,	0.3323,	0.3555,	0.3139,	-0.1971,	-0.0703,	
-0.5721,	0.5277,	0.2169,	0.2295,	0.0806,	-0.0821,	-0.0810,	-0.0473,	0.0258,	0.2180,	-1.0748,	
-0.3741,	-0.4136,	-0.2667,	0.0858,	-0.1651,	0.4323,	-0.6172,	0.3687,	0.5480,	0.1269,	0.4493,	
-0.1455,	-0.6407,	0.5454,	-0.4102,	-0.3061,	0.0469,	0.3951,	0.2586,	-0.2073,	-0.1485,		

Problem 2: Character-Level Name Generation — Architecture & RNN-Based Architecture

2.1 Task Description & Dataset

The task is to learn the character-level distribution of Indian names and generate novel, realistic names autoregressively. A model is trained to predict the next character given all preceding characters: $P(c_t | c_1, \dots, c_{t-1})$. During generation, characters are sampled from the predicted distribution (temperature-controlled softmax) and fed back as the next input, continuing until $\langle \text{EOS} \rangle$ is emitted.

Dataset: 1,000 unique Indian names curated from North Indian, South Indian, Bengali, Gujarati, Marathi, and pan-Indian pools. Names generated programmatically from curated lists of real names supplemented by constructed names using common Indian phoneme prefixes and suffixes. Average name length: 6.1 characters; range: 2-16.

2.2 Model 1 — Vanilla RNN

A Vanilla RNN computes hidden states using the recurrence relation: $h_t = \tanh(W_{ih} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$. The tanh non-linearity squashes outputs to $[-1, 1]$. With 2 stacked layers, the output of layer l feeds as input to layer $l+1$. The final hidden state projects through a linear layer to logits over the 49-character vocabulary.

Architecture Embedding (49×32) $\rightarrow$ $2 \times$ VanillaRNNCell (hidden_size=128) $\rightarrow$ Linear (128 $\rightarrow$ 49)

Training Teacher-forcing: ground-truth character fed as input at each timestep; cross-entropy vs. next-character target.

Weakness Suffers from vanishing gradients for long sequences (tanh derivatives < 1 compound multiplicatively over timesteps), but less severe here since Indian names are short (avg 6 chars).

2.3 Model 2 — Bidirectional LSTM (Bi-LSTM)

LSTM addresses vanishing gradients via gating: the cell state c_t flows with only element-wise multiplication, preserving gradients. Four gates computed simultaneously from $[h_{t-1}, x_t]$:

```
i_t = σ(W_i · [h_{t-1}, x_t] + b_i)  ← Input gate: what to write
f_t = σ(W_f · [h_{t-1}, x_t] + b_f)  ← Forget gate: what to erase
g_t = tanh(W_g · [h_{t-1}, x_t] + b_g) ← Candidate values
o_t = σ(W_o · [h_{t-1}, x_t] + b_o)  ← Output gate: what to expose
c_t = f_t ⊙ c_{t-1} + i_t ⊙ g_t
h_t = o_t ⊙ tanh(c_t)
```

Bidirectionality adds a backward LSTM running from right to left over the sequence. Forward and backward hidden states are concatenated at each timestep, doubling the representation width ($2 \times 128 = 256$). This allows the model to leverage both past and future context during training (teacher-forcing), though inference uses forward-only pass since future characters are unavailable.

2.4 Model 3 — RNN + Luong Attention

Attention allows the decoder to selectively focus on relevant past hidden states when predicting each character. At timestep t , Luong general attention computes:

```
score(h_t, h_j) = h_t^T · W_attn · h_j    for all j < t
α = softmax(scores)                       attention weights
context_t = Σ_j α_j · h_j                 weighted combination
output_t = tanh(W_c · [context_t ; h_t])  combined representation
```

The attention weight α_j represents how much the model 'attends' to character position j when generating position t . This is particularly useful for learning recurring patterns in names, such as common suffixes (-esh, -raj, -iya) or vowel placement conventions.

2.5 Training Configuration

Hyperparameter	Vanilla RNN	Bi-LSTM	Attention+RNN
Embedding dim	32	32	32
Hidden size	128	128	128
Layers	2	2	2
Dropout	0.20	0.40	0.30
Learning rate	0.003	0.003	0.003
Epochs	100	60	60
LR scheduler	StepLR (γ=0.5, step=30)StepLR (γ=0.5, step=20)StepLR (γ=0.5, step=20)		
Final training loss	1.137	0.003	1.510
Parameters	61,393	442,193	271,185
Checkpoint size	0.24 MB	1.69 MB	1.04 MB

2.6 Quantitative Evaluation (500 generated names each)

Model	Novelty Rate	Diversity	Params
Vanilla RNN	24.6%	80.4%	61,393
Bi-LSTM	100.0%	97.6%	442,193
Attention+RNN	100.0%	100.0%	271,185

Novelty Rate = percentage of generated names not present in the training set. Diversity = unique generated names / total generated.

2.7 Qualitative Analysis — Sample Generated Names

Vanilla RNN Chaitanya, Namit, Vasuki, Sahan, Shubh, Devi, Bhina, Pashay, Sakar — realistic, pronounceable Indian names.

Bi-LSTM Ojw, Yegi, Tookst, Yotas, Gokgs, Hgoy — novel but largely non-Indian character strings. High diversity but low realism.

Attention+RNN Highly diverse but structurally inconsistent strings. Names follow no clear phonotactic pattern of Indian names.

2.8 Discussion — Why Vanilla RNN Works Best Here

The central finding is counterintuitive: the simplest model produces the most realistic names. The explanation lies in the interplay of model capacity and training set size:

Capacity-Data Mismatch Bi-LSTM (442K params) has extreme capacity relative to 1,000 short training sequences. During teacher-forcing training, it memorises the exact training distribution (loss → 0.003). At inference time, the autoregressive loop introduces small distribution shifts that cascade into incoherent outputs — the model has never seen its own mistakes.

Bi-directionality at Inference The backward LSTM is a key structural issue: backward context (future characters) is available during training but fundamentally absent during generation. The model is trained with more information than it has at test time, creating a train-test mismatch.

Attention on Short Sequences Luong attention is most useful when sequences are long enough to contain diverse, position-specific patterns to attend to. With average length 6 characters, there are at most 5 previous positions to attend — the attention mechanism adds parameters without providing proportional representational benefit.

Vanilla RNN's Advantage With only 61K parameters, the Vanilla RNN cannot memorise individual training names. Instead, it is forced to generalise: it learns the statistical regularities of Indian phonology — common consonant clusters, vowel patterns, and name-termination cues — which transfer well to generation.

Novelty vs. Realism Trade-off High novelty (100%) is not inherently better; it merely means the model never exactly reproduces training names. If those novel names are structurally invalid (no vowels, random consonants), novelty is a misleading metric. Realism requires learning the underlying phonotactics, which the Vanilla RNN does most effectively here.

2.9 Common Failure Modes

Failure Mode	Vanilla RNN	Bi-LSTM	Attention+RNN
Too short (< 2 chars)	0	5	2
Repetitive patterns	3	4	15
No vowels	0	25	84
Total failures (500 samples)	3	34	101

The 'no vowels' metric is particularly diagnostic: it indicates the model has not internalised the universal constraint that Indian names contain vowels. Vanilla RNN produces zero such cases; Bi-LSTM and Attention+RNN produce 25 and 84 respectively — a direct consequence of over-fitting and train-inference mismatch.

3.0 Project-Level Conclusions

► Problem 1 — Key Takeaways

- Building a diverse corpus (Annual Reports, newsletters, brochures) is as important as model architecture. The 6× expansion from 0.17 to 0.94 MB significantly improved embedding quality and vocabulary.
- CBOW (dim=200, win=5) achieves the best average-similarity score (0.855) by smoothing context. Skip-gram excels at analogy resolution by learning richer directional relationships.
- The from-scratch PyTorch implementation reproduces the qualitative neighbourhood structure of Gensim with the same training data, validating the correctness of the implementation.
- Domain-specific vocabulary (viva-voce, cr, dugc, mtech) poses a challenge for analogy evaluation because standard evaluation words may not appear in the vocabulary. This was handled by selecting in-vocabulary analogy triplets.

► Problem 2 — Key Takeaways

- Model complexity should be matched to dataset size. For short sequences and small datasets, simpler models generalise better.
- Teacher-forcing trains models under assumptions that break at inference time (perfect previous characters); this 'exposure bias' is more harmful for high-capacity models.
- Bidirectionality is architecturally incompatible with autoregressive generation — a structural mismatch that fundamental limits Bi-LSTM generation quality regardless of training completeness.
- Attention mechanisms are most beneficial for long-range dependencies; they add overhead on short sequences relative to their benefit.
- Quantitative metrics (novelty, diversity) and qualitative realism can diverge significantly. A holistic evaluation framework is essential for sequence generation tasks.

3.1 Summary Table

Metric	P1 CBOW (best)	P1 Skip-gram (best)	P2 Vanilla RNN
NLP Assignment 2 · Jateen (B22CS026) · IIT Jodhpur			

Corpus / Dataset	0.937 MB / 50 docs	0.937 MB / 50 docs	1,000 names
Parameters	CBOW d=200	SG d=50	61,393
Best score	avg_sim=0.855	avg_sim=0.735	Novelty 24.6%
Key strength	Tight clusters	Analogy tasks	Realism